

Loom Recording Script

Project: Debales AI - Webhook Automation Engine

Target Length: 8 minutes

Format: Screen + voice recording

Before You Start

Do these steps before hitting record:

1. Wipe the database so you start completely clean:

```
docker-compose down -v  
docker-compose up -d --build
```

2. Wait 15 seconds for everything to boot up
 3. Open your browser at **http://localhost:5173**
 4. Open VS Code with
`backend/src/common/interceptors/idempotency.interceptor.ts`
loaded in a tab
 5. Have a terminal window ready in the project root folder
 6. Arrange your screen: browser on the right, terminal on the left
-

Part 1 — Introduction (0:00 to 0:30)

Screen to show: Dashboard homepage at `http://localhost:5173`

What to say:

"Hi, I'm going to walk you through the Webhook Automation Engine I built. The app is already running using Docker, and you can see the dashboard right here.

I'll show you four things — a webhook going through the full pipeline, the deduplication guard in action, a failure with automatic retries, and then a manual replay.

Let's get into it."

Part 2 — Sending a Webhook and Watching It Process (0:30 to 2:30)

Screen to show: Terminal + Dashboard (switch between them)

Step 1 — Run the script

Switch to the terminal and run:

```
.\scripts\send-webhook.ps1 -TenantId 6a460515c610ca63064b6027
```

What to say while running:

"I'll start by sending a real webhook. This simulates a Shopify order coming in for \$750. Company A — Acme Corp — has a rule that fires on any order over \$500."

What to say after you see the terminal response:

"You can see the terminal printed a response in under 100 milliseconds — that's the 200 OK going back to the caller. The webhook was accepted and queued immediately. Nothing was processed yet — all the heavy work happens in the background."

Step 2 — Switch to the Events tab on the dashboard

What to say:

"On the Events tab, I can see the event came in from Shopify, the type is 'order.created', and it's already marked as 'processed'. That happened asynchronously — the API responded immediately, and the worker picked it up in the queue right after."

Step 3 — Click the Jobs tab

What to say:

"The Jobs tab is where it gets interesting. One single webhook triggered three separate jobs — one for each action the rule was configured to run. An email, a log, and a webhook to an external API. Each one ran independently in its own queue slot."

This matters because if the email fails, the webhook still completes. If the external API is slow, the log action doesn't have to wait for it. This is fault isolation by design."

What to say while pointing at the Attempts column:

"Each row shows Attempt: 1 — everything worked on the first try. Let me click View Details on one of these."

Step 4 — Click "View Details" on the email job

What to say:

"Here you can see the full execution timeline — when it started, when it finished, and what the result was. Every single job stores this. You have a complete audit trail for everything that ran."

Part 3 — Deduplication (2:30 to 4:00)

Screen to show: Terminal first, then switch to Dashboard Events tab

Step 1 — Explain the problem

What to say:

"Now I'll show deduplication. Platforms like Shopify re-send webhooks automatically if they don't get a fast response. This is a real-world problem — you don't want to charge a customer twice or send two confirmation emails for one order."

Step 2 — Run the duplicate script

```
.\scripts\send-duplicate.ps1 -TenantId 6a460515c610ca63064b6027
```

What to say while pointing at the terminal output:

"This script sends the exact same webhook twice, one second apart. Watch — the first one says 'status: accepted'. The second one says 'status: duplicate'. It was caught before it even touched the database."

Step 3 — Switch to Events tab

What to say:

"On the Events tab, there is only one new row — not two. And if you look at the Jobs tab, the count went up by 3, not 6. The second webhook was dropped at the Redis layer — atomically, with no race conditions possible."

Part 4 — Failure, Retries, and Replay (4:00 to 6:30)

Screen to show: Dashboard (switch tenant to Beta Store), Terminal, Jobs tab

Step 1 — Switch tenant

Switch the dropdown at the top right from "Acme Corp" to "Beta Store".

What to say:

"For this test I'm switching to Company B — Beta Store — using the dropdown at the top right."

Step 2 — Run the failure script

```
.\scripts\trigger-failure.ps1 -TenantId 6bb71626d721db74175c7138
```

What to say:

"Beta Store's rule points to an external API that always returns a 500 error. I've set this up on purpose to show how the system handles failures."

Step 3 — Switch to Jobs tab and watch the status**What to say while watching the status change:**

"Watch the status column. It starts as 'pending', moves to 'retrying' after the first failure, retries again with a delay, and eventually lands on 'failed' after three attempts."

It doesn't crash. It doesn't silently drop the job. It records everything and keeps going."

Step 4 — Click "View Details" on the failed job**What to say:**

"In the detail view I can see each attempt individually — the exact timestamp, the error message the external API returned, and when the retry delay kicked in. This is stored as an array inside the job document itself, so one database query gives you the full history. No joins, no extra lookups."

Step 5 — Click "Replay This Job"**What to say:**

"Now I'll hit Replay. The system takes the exact original payload from when this event first arrived — the actual stored payload, not a copy from memory — and sends it through the pipeline again from scratch."

Watch the Jobs tab. A new row appears. That's the replay running with its own retry cycle."

Part 5 — The Decision I'm Most Proud Of (6:30 to 8:00)

Screen to show: VS Code with `idempotency.interceptor.ts` open

Step 1 — Open the file

Open: `backend/src/common/interceptors/idempotency.interceptor.ts`

Scroll to the `intercept()` method and the Redis SETNX line.

What to say:

"Let me walk you through the one piece of code I'm most proud of — the `IdempotencyInterceptor`."

Step 2 — Point at the intercept method

What to say:

"When a webhook hits the endpoint, this interceptor runs first — before the controller, before any database write. It builds a key from the tenant ID, the source, and the event ID, and fires a single atomic Redis operation: SET if Not Exists, with a 24-hour expiry.

If the key already exists, it means we've seen this event before. We return 200 immediately and skip all processing. We don't even enter the controller."

Step 3 — Explain why returning 200 matters

What to say:

"Returning 200 is important here. External platforms like Shopify stop retrying when they see a 200. If I returned a 409 or 400 for a duplicate, they'd keep hammering the endpoint.

The response body says 'status: duplicate' so the caller knows what happened, but the HTTP status is still 200."

Step 4 — Explain the interceptor choice

What to say:

"I implemented this as a NestJS interceptor rather than inside the service because deduplication is a cross-cutting concern. It doesn't belong in business logic. It runs in the request lifecycle, cleanly separated, and is completely transparent to everything downstream."

Step 5 — Mention the MongoDB backup layer

What to say:

"There's also a unique index on the idempotency key in MongoDB as a secondary safety net — in case Redis restarts and loses recent state. Two layers of deduplication. Neither one alone is sufficient for production."

Part 6 — Close (8:00 to 8:30)

Screen to show: Back to the Dashboard homepage

What to say:

"That's the full demo. The system handles reliable ingestion, deduplication, async processing, failure recovery, manual replay, and full observability — all from this one

dashboard.

The README covers all the setup steps and architecture decisions in detail if you want to read through it. Thanks for watching."

Quick Reference — Commands Used

Part	Command
Clean start	<code>docker-compose down -v</code> then <code>docker-compose up -d --build</code>
Test 1 (happy path)	<code>.\scripts\send-webhook.ps1</code> <code>-TenantId</code> <code>6a460515c610ca63064b6027</code>
Test 2 (deduplication)	<code>.\scripts\send-duplicate.ps1</code> <code>-TenantId</code> <code>6a460515c610ca63064b6027</code>
Test 3 (failure + replay)	<code>.\scripts\trigger-failure.ps1</code> <code>-TenantId</code> <code>6bb71626d721db74175c7138</code>

Screen Cheat Sheet

Time	What to show
0:00 - 0:30	Dashboard homepage
0:30 - 0:45	Terminal (run send-webhook)
0:45 - 1:15	Dashboard - Events tab
1:15 - 2:00	Dashboard - Jobs tab
2:00 - 2:30	Dashboard - Job Detail view
2:30 - 2:45	Terminal (run send-duplicate)
2:45 - 4:00	Dashboard - Events + Jobs tab
4:00 - 4:10	Dashboard - Switch to Beta Store
4:10 - 4:20	Terminal (run trigger-failure)
4:20 - 5:30	Dashboard - Jobs tab (watch retrying)
5:30 - 6:00	Dashboard - Job Detail view (attempts)
6:00 - 6:30	Dashboard - Replay button + new job row
6:30 - 8:00	VS Code - idempotency.interceptor.ts
8:00 - 8:30	Dashboard homepage

